

PROG. GESTIONE DI UN COMUNE (3)

Il documento mostra le **scelte progettuali e l'implementazione del programma** per la gestione delle segnalazioni di un Comune. Il software ha l'obiettivo di memorizzare, ricercare, filtrare e gestire in modo efficiente le varie segnalazioni inviate dai cittadini (es. incidenti, problematiche, etc.), garantendo tempi di risposta rapidi anche con un elevato volume di dati.

2. Scelte Progettuali e Strutture Dati

Per garantire l'efficienza del sistema, si è posta particolare attenzione alla scelta delle strutture dati, bilanciando la necessità di un accesso rapido alle informazioni con un'occupazione dinamica e ottimizzata della memoria.

2.1 La Tabella Hash

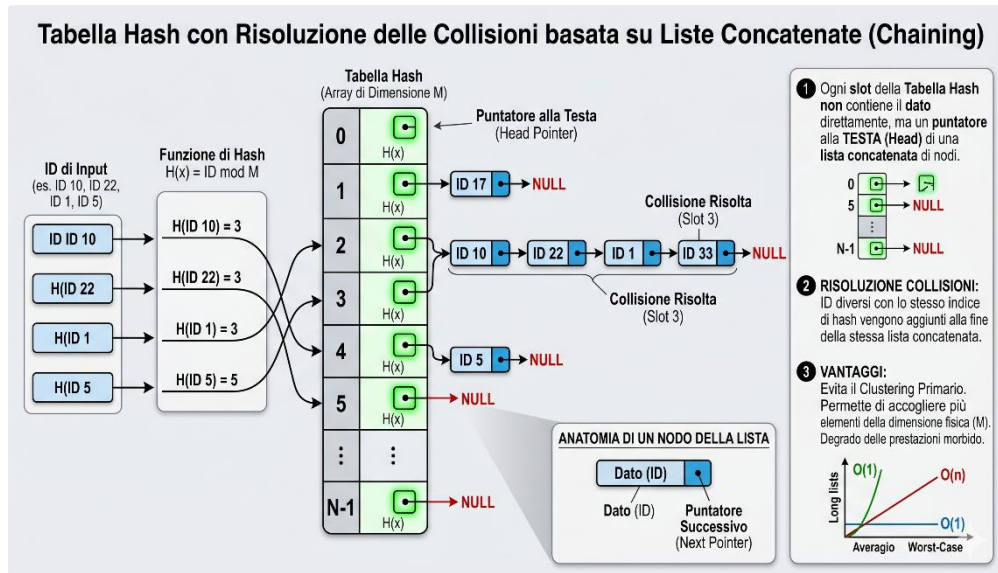
La struttura dati principale scelta per memorizzare le segnalazioni è la **Tabella Hash**. Questa decisione è stata motivata dalle seguenti necessità operative del sistema comunale:



- **Ricerca istantanea:** L'operatore deve poter rintracciare una segnalazione specifica conoscendone l'ID in tempi minimi. La tabella hash, tramite la funzione di hashing $hashFun(key, size)$, garantisce un tempo di accesso medio pari a $O(1)$.
- **Inserimento rapido:** L'aggiunta di nuove pratiche avviene in tempo costante, rendendo il sistema scalabile anche durante i picchi di segnalazioni.
- **Flessibilità:** Rispetto a un array statico, la tabella hash permette di gestire moli di dati imprevedibili in modo molto più strutturato.

2.2 Gestione delle Collisioni: Indirizzamento Concatenato

Poiché la funzione di hashing può generare lo stesso indice per ID diversi e avere così una collisione, si è scelto di implementare una risoluzione basata su **liste concatenate**, unendo così uno degli argomenti fatti a inizio lezioni con uno degli ultimi svolti.



- Ogni **slot** della tabella hash non contiene direttamente il dato, ma un puntatore alla testa di una lista concatenata di nodi.
- Questo approccio evita il problema del clustering primario tipico dell'indirizzamento aperto e permette alla tabella di accogliere un numero di elementi superiore alla sua dimensione fisica, con un degrado delle prestazioni morbido nel caso peggiore ($O(n)$ sulla singola lista).

2.3 Astrazione Modulo Item

Per rendere il codice **modulare e riutilizzabile**, le informazioni specifiche del dominio sono state isolate nel modulo `item.c` e `item.h`.

La tabella hash e le liste non conoscono i dettagli interni della Segnalazione, ma vi accedono tramite funzioni interfaccia come `get_id_segnalazione()` o `get_stato_segnalazione()`. Questo garantisce un ottimo livello di incapsulamento.

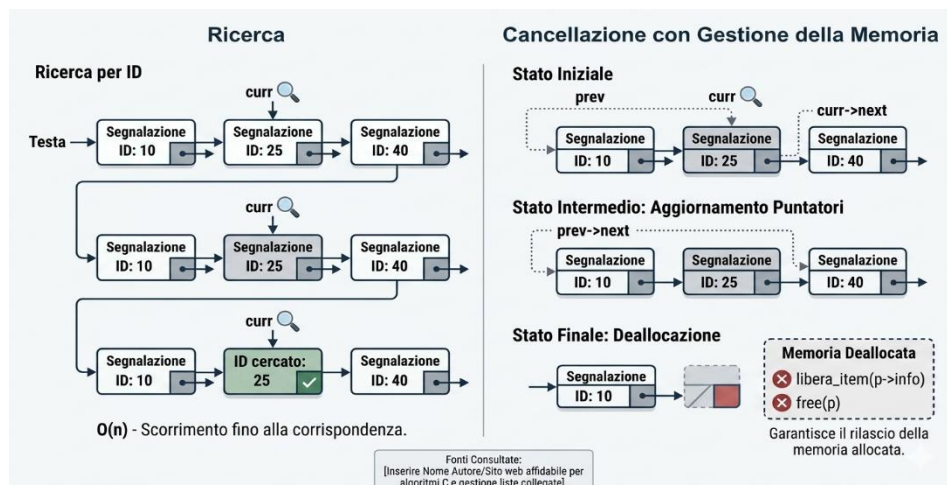
3. Implementazione dei Moduli

Il progetto è suddiviso in moduli distinti, ognuno con una singola responsabilità, facilitando la manutenzione e il testing del codice.

3.1 Modulo Lista

Il modulo lista fornisce le operazioni base per manipolare le catene di collisioni:

- **Inserimento in testa:** Ottimale per le operazioni di chaining, con costo $O(1)$.
- **Ricerca e Cancellazione per ID:** Funzioni che scorrono i nodi aggiornando correttamente i puntatori per prevenire **memory leak**.



L'immagine mostra i tre passaggi per eliminare in sicurezza un nodo (segnalazione) all'interno della struttura dati, garantendo l'integrità dei collegamenti e prevenendo i memory leak:

1. **Fase di Ricerca:** Il sistema utilizza un puntatore corrente (*curr*) per scorrere la lista partendo dalla testa, confrontando l'ID cercato con quello contenuto in ogni nodo. Contestualmente, un puntatore ausiliario (*prev*) tiene traccia del nodo appena visitato.
2. **Aggiornamento dei Puntatori:** Una volta individuato il nodo target, il collegamento logico viene modificato. Il puntatore *next* del nodo precedente viene riassegnato in modo da puntare direttamente al nodo successivo a quello da eliminare ($prev \rightarrow next = curr \rightarrow next$) mantenendo intatta la continuità della lista.
3. **Deallocazione della Memoria:** Il nodo, occupa ancora spazio nella memoria heap. Per evitare un memory leak, creo la distruzione tramite la funz. `libera_item` e infine la deallocazione fisica del nodo stesso tramite la direttiva `free(curr)`.

3.2 Modulo Hash

Il modulo principale espone la gestione dell'intero database

- **InsertHash:** Calcola l'indice, verifica l'assenza di duplicati scorrendo la lista corrispondente, e in caso negativo inserisce il nuovo elemento.

```
26 int InsertHash(hashtable h, Item elem) {
27     int idx;
28     struct Nodo *head, *curr;
29     int key = get_id_item(elem);
30
31     idx = hashFun(key, h->size);
32     curr = head = h->table[idx];
33
34     while (curr) {
35         if (get_id_item(curr->info) == key) {
36             return (0);
37         }
38         curr = curr->next;
39     }
40
41     struct Nodo* nuovo = (struct Nodo*) malloc(sizeof(struct Nodo));
42     nuovo->info = elem;
43     nuovo->next = head;
44
45     h->table[idx] = nuovo;
46     return (1);
47 }
```

- **HashDelete:** Cerca l'elemento per chiave e lo rimuove. Ripristina correttamente il collegamento tra il nodo precedente e il successivo, liberando la memoria allocata dinamicamente con free().

```
61 int HashDelete(hashtable h, int key) {
62     int idx;
63     struct Nodo *prev, *curr, *head;
64
65     idx = hashFun(key, h->size);
66     prev = curr = head = h->table[idx];
67
68     while (curr) {
69         if (get_id_item(curr->info) == key) {
70             if (curr == head) h->table[idx] = curr->next;
71             else prev->next = curr->next;
72
73             libera_item(curr->info);
74             free(curr);
75             return 1;
76         }
77         prev = curr;
78         curr = curr->next;
79     }
80     return 0;
81 }
```

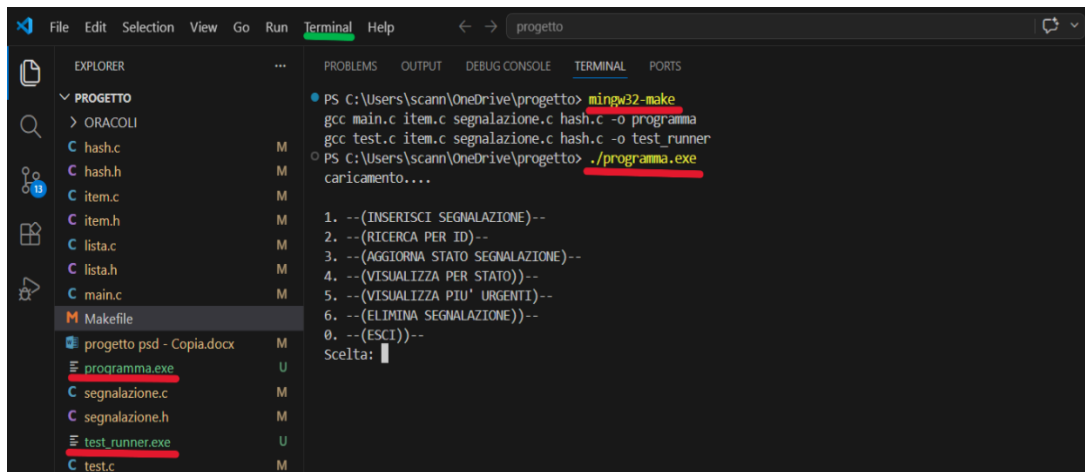
- **Funzioni di utilità:** Sono state implementate funzioni avanzate per le esigenze del Comune, come stampa_per_stato_tabhash e stampa_piu_urgenti_tabhash, che iterano sull'intera struttura per generare report filtrati.

```
01 void stampa_per_stato_tabhash(hashtable h, int stato) {
02     if (!h) return;
03     for (int i = 0; i < h->size; i++) {
04         struct Nodo* curr = h->table[i];
05         while (curr != NULL) {
06             if (get_stato_item(curr->info) == stato) {
07                 stampa_item(curr->info);
08             }
09             curr = curr->next;
10         }
11     }
12 }
13
```

4. Come interagire con il Sistema

Il sistema è stato progettato per essere compilabile tramite il **Makefile** fornito a corredo del codice sorgente.

- **Compilazione:** Per compilare l'intero progetto, è sufficiente aprire il terminale nella directory principale e digitare il comando `mingw32-make`. Questo si occuperà di compilare i singoli moduli (`item.c`, `lista.c`, `hash.c`, etc.) e di generare il file eseguibile finale.
- **Esecuzione:** Una volta terminata la compilazione, il programma può essere avviato eseguendo il file generato da terminale (`./programma.exe`). Il sistema leggerà le segnalazioni dai file di input previsti e stamperà i risultati, oppure permetterà l'interazione per cercare, inserire o eliminare le pratiche.
- **Pulizia:** Per mantenere la directory di lavoro pulita, è possibile utilizzare il comando `make clean`, che rimuoverà in automatico tutti i file oggetto e l'eseguibile, riportando la cartella allo stato iniziale.



The screenshot shows the Visual Studio Code interface. The Explorer view on the left displays the project structure with files like `hash.c`, `item.c`, `lista.c`, `main.c`, `segnalazione.c`, `segnalazione.h`, `test.c`, `programma.exe`, `test_runner.exe`, and `Makefile`. The Terminal view on the right shows the execution of `mingw32-make` and `./programma.exe`. The output of the program is displayed in the terminal, showing a menu with options 1 through 6 and 0, and a prompt for selection.

```
PS C:\Users\scann\OneDrive\progetto> mingw32-make
gcc main.c item.c segnalazione.c hash.c -o programma
gcc test.c item.c segnalazione.c hash.c -o test_runner
PS C:\Users\scann\OneDrive\progetto> ./programma.exe
caricamento....

1. --(INSERISCI SEGNALAZIONE)--
2. --(RICERCA PER ID)--
3. --(AGGIORNA STATO SEGNALAZIONE)--
4. --(VISUALIZZA PER STATO)--
5. --(VISUALIZZA PIU' URGENTI)--
6. --(ELIMINA SEGNALAZIONE)--
0. --(ESCI)--
Scelta: 
```

5. Ambiente di Sviluppo

L'intero progetto con il codice in C è stato interamente sviluppato e testato in ambiente **Windows**. L'ambiente di sviluppo integrato utilizzato per la scrittura, la gestione dei moduli e il debugging è stato **Visual Studio Code**.

6. Spec. Sintattica e semantica

ADT ITEM

SINTATTICA

- **Tipo di riferimento:** Item
- **Tipi usati:** int, void
- **Operatori:**
 - get_id_item(Item) -> int
 - get_stato_item(Item) -> int
 - get_urgenza_item(Item) -> int
 - stampa_item(Item) -> void
 - libera_item(Item) -> void

SEMANTICA

- get_id_item(i) -> k
 - Post: restituisce l'intero k che rappresenta la chiave dell'item i.
- get_stato_item(i) -> s
 - Post: restituisce l'intero s che rappresenta lo stato dell'item i.
- get_urgenza_item(i) -> u
 - Post: restituisce l'intero u che rappresenta il livello di urgenza dell'item i.
- stampa_item(i) -> void
 - Post: stampa a video le informazioni contenute nell'item i.
- libera_item(i) -> void
 - Post: l'area di memoria allocata per l'item i viene deallocata.

ADT LISTA

SINTATTICA

- **Tipo di riferimento:** Lista* (o Lista)

- **Tipi usati:** Item, int, void

- **Operatori:**

- crea_lista() -> Lista*
- inserisci_in_testa(Lista*, Item) -> void
- cerca_per_id(Lista*, int) -> Item
- elimina_per_id(Lista*, int) -> int
- distruggi_lista(Lista*) -> void

SEMANTICA

- crea_lista() -> l

- Post: crea e restituisce una lista vuota.

- inserisci_in_testa(l, e) -> void

- Post: l' = l'elemento e diventa il primo della sequenza.

- cerca_per_id(l, k) -> e

- Pre: l = <a_1, a_2, ..., a_n>
- Post: e = a_i se esiste un elemento in l con id uguale a k, e = NULL altrimenti.

- elimina_per_id(l, k) -> b

- Pre: l = <a_1, ..., a_n>
- Post: se esiste a_i con id uguale a k, l' = <a_1, ..., a_{i-1}, a_{i+1}, ..., a_n> e restituisce 1. Se non esiste, l' = l e restituisce 0.

- distruggi_lista(l) -> void

- Post: la lista l e tutti i nodi al suo interno vengono completamente rimossi dalla memoria.

ADT HASH

SINTATTICA

- **Tipo di riferimento:** hashtable

- **Tipi usati:** Item, int, void

- **Operatori:**

- newHashtable(int) -> hashtable
- InsertHash(hashtable, Item) -> int
- SearchHash(hashtable, int) -> Item
- HashDelete(hashtable, int) -> int
- DestroyHashtable(hashtable) -> void
- stampa_per_stato_tabhash(hashtable, int) -> void
- stampa_piu_urgenti_tabhash(hashtable) -> void

SEMANTICA

- newHashtable(size) -> t

- Post: $t = \{\}$ (crea una tabella hash vuota con capacità logica size).

- InsertHash(t, e) -> b

- Pre: e deve essere un item valido.
- Post: se la chiave di e non è presente, restituisce 1. Se la chiave è già presente la tabella non cambia e restituisce 0.

- SearchHash(t, k) -> e

- Pre: $t = (a_1, \dots, a_n)$.
- Post: $e = a_i$ con i compreso tra 1 e n, la chiave di a_i è k; altrimenti $e = \text{NULL}$.

- HashDelete(t, k) -> b

- Pre: $t = (a_1, \dots, a_n)$.
- Post: se esiste a_i con chiave k, l'elemento viene rimosso e deallocato e restituisce 1. Se non esiste restituisce 0.

- DestroyHashtable(t) -> void

- Post: l'intera struttura t e tutti gli item in essa contenuti vengono completamente distrutti e deallocati.

- stampa_per_stato_tabhash(t, s) -> void

- Pre: $t = (a_1, \dots, a_n)$.
- Post: vengono stampati a video tutti gli a_i appartenenti a T per i quali $\text{get_stato_item}(a_i) == s$.

- stampa_piu_urgenti_tabhash(t) -> void

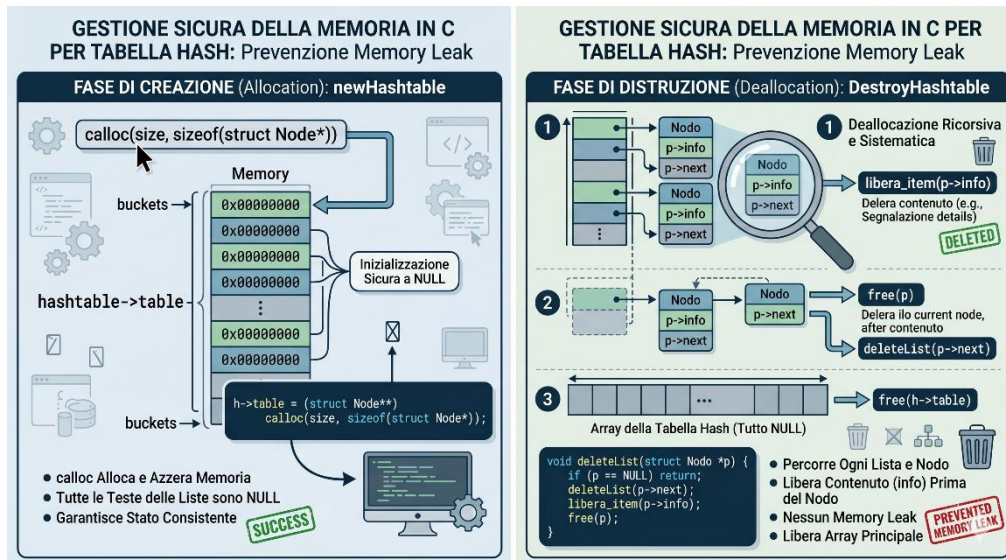
- Pre: $t = (a_1, \dots, a_n)$.
- Post: vengono stampati a video tutti gli a_i appartenente a T per i quali il livello di urgenza è uguale al massimo livello di urgenza presente nell'insieme t .

7. Casi di Test

- **TC1 ->** il file di input contiene 10 e l'oracolo 1, Il programma legge l'ID 10. Calcola l'indice: $10 \% 5 = 0$. Inserisce la segnalazione nello slot 0. Poi, cerca l'ID 10. Lo trova, quindi scrive 1 nel file output.txt. Infine, confronta l'output (1) con l'oracolo (1). Essendo uguali, stampa **PASS**.
- **TC2 ->** il file di input contiene 1, 2, 3, 4. e l'oracolo 1, 1, 1, 1. Verifica che la tabella sappia gestire più elementi che si vanno a posizionare in slot diversi senza disturbarsi a vicenda. Il programma legge i 4 numeri e calcola i loro indici Nessun elemento sbatte contro un altro. Il programma li cerca tutti e 4, li trova al primo colpo, scrive quattro 1 nell'output. Confronta con l'oracolo e stampa **PASS**.
- **TC3 ->** il file di input contiene 5, 10, 15, 20, 25. e l'oracolo 1, 1, 1, 1, 1. Verifica che il codice per il chaining funzioni perfettamente. Se la gestione delle liste ha un difetto, questo test fallisce. Tutti e cinque i numeri sono multipli di 5, tutti indice 0. Finiscono tutti nello slot 0, che diventa una lista concatenata lunga 5 elementi. Quando il programma chiama SearchHash, deve scorrere tutta la lista per trovare gli elementi. Se li trova tutti, l'output sarà tutto 1, l'oracolo combacia e stampa **PASS**.
- **TC4 ->** il file di input contiene 7, 8. e l'oracolo 0, 1. Simuliamo che l'oracolo sia sbagliato. Il programma inserisce normalmente il 7 e l'8, e riesce a trovarli entrambi. Il file output conterrà 1 e 1, ma quando va a leggere l'oracolo per fare il confronto, trova 0 e 1. La funzione confronta array nota che il primo numero è diverso. Si ferma e stampa **FAIL**. Così sappiamo che il controllo funziona.
- **TC5 ->** il file di input contiene 11, 23, 31, 44, 57, 66, 79, 80 e l'oracolo 1, 1, 1, 1, 1, 1, 1, 1. Gli elementi verranno sparsi in modo pseudo-casuale nei vari slot. Alcuni slot avranno un solo elemento, altri avranno liste concatenate con collisioni. Il programma deve riuscire a recuperarli tutti senza confondersi. Se lo fa, stampa **PASS**.

8. Gestione della Memoria

Essendo il C un linguaggio a gestione manuale della memoria, si è posta massima attenzione alla prevenzione di **memory leak**:



- In fase di creazione con `newHashtable`, si utilizza **calloc** per garantire che tutti i puntatori in testa alle liste siano inizializzati a NULL in modo sicuro.
- In fase di distruzione con `DestroyHashtable`, una funzione ricorsiva si occupa di percorrere ogni singola lista, **deallocando** prima il contenuto informativo con `libera_item` e successivamente il nodo stesso, prima di liberare l'array principale della tabella.

9. Conclusioni

Personalmente sono convinto che la combinazione di una Tabella Hash supportata da Liste Concatenate si è rivelata la soluzione algoritmica ideale per i requisiti del progetto. Il sistema garantisce tempi di risposta ottimali per le operazioni quotidiane e mantiene un'impronta di memoria efficiente, risultando uno strumento affidabile per la gestione delle pratiche comunali. Tutto ciò rende questo programma una **piattaforma solida e affidabile** per la gestione delle pratiche comunali, capace di garantire continuità operativa e di adattarsi facilmente a futuri incrementi della mole di dati.